

Clase 6

Tema 4: Caracterización de modelos
computacionales de redes neuronales (Deep
Learning)
Parte 1

Ciclo: Máster en IA y Big Data
Módulo: MP2: Sistemas de aprendizaje
automático

Nil Redón Orriols

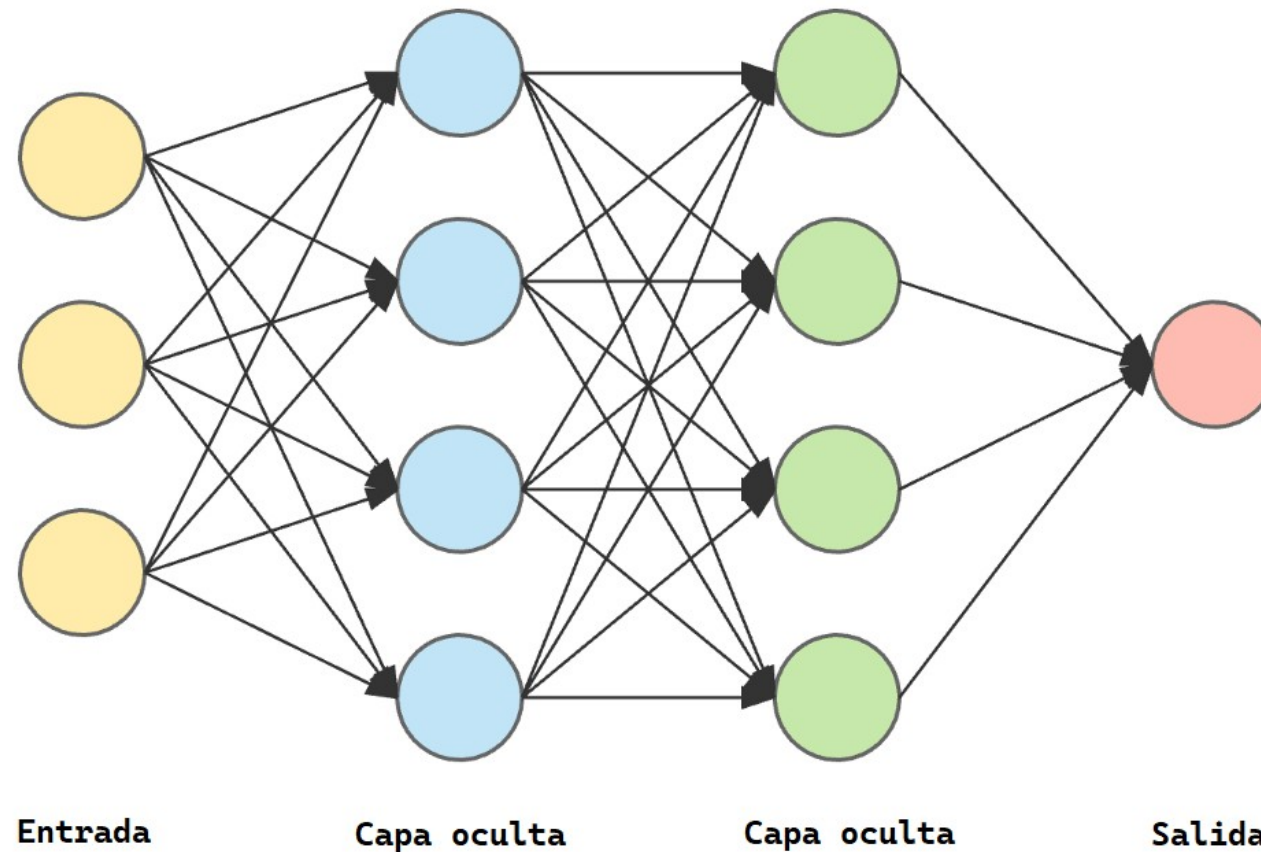
Contenidos de la clase de hoy

- Definición y características de redes neuronales
- Proceso de entrenamiento de redes neuronales

Definición de una red neuronal

- Es un modelo de aprendizaje automático que imita el funcionamiento del cerebro humano.
- Está compuesta por nodos que se organizan en capas y procesan y transfieren información.
- El proceso se repite hasta que se obtiene una salida final como resultado.

Componentes de una red neuronal



Componentes de una red neuronal

- Capa de entrada: Primera capa de la red neuronal, recibe los datos de entrada del algoritmo
- Capas ocultas: Capas intermedias entre la capa de entrada y la capa de salida, es donde se realiza el procesamiento abstracto de los datos.
- Capa de salida: Última capa de la red neuronal, produce la salida final del modelo.

Componentes de una red neuronal

Capa de entrada:

- No realiza cálculos, sólo distribuye los datos a las capas siguientes.
- Cada entrada representa una característica de los datos, convertida a número
- Ejemplo, si procesamos un dataset con 50 columnas, la capa de entrada tiene dimensión 50 (50 neuronas).

Componentes de una red neuronal

Capas ocultas:

- No hay un número fijo de capas intermedias ni de neuronas, puede haber infinitos diseños y combinaciones.
- Según el tipo de capa oculta, definiremos el modelo de red neuronal (FNN, CNN, RNN...)
- Es la caja negra donde se procesan los datos y el modelo aprende patrones ocultos en los mismos.

Componentes de una red neuronal

Capa de salida:

- Muestra el resultado final del procesamiento en las capas ocultas.
- La cantidad de neuronas en la capa depende del tipo de problema que se esté resolviendo. Por ejemplo:
 - Clasificación binaria: 1 neurona
 - Clasificación multiclase: 1 neurona por clase
 - Regresión: 1 neurona

Componentes de una red neuronal

Una capa se forma de neuronas conectadas entre ellas, cada una de las cuales recibe datos del paso anterior y ejecuta un proceso secuencial que consta de tres pasos:

- Recepción de los inputs: Valores numéricos que corresponde a la salida de la capa anterior.
- Pesos, suma ponderada y sesgo: Transformación matemática sobre las entradas recibidas.
- Función de activación: Función matemática sobre el valor obtenido en el paso anterior.

Componentes de una red neuronal

Paso 1: Recepción de los inputs:

- Conjunto de datos que vienen del paso anterior.
- La neurona recibe un único valor numérico de cada neurona de la capa anterior con la que está conectada.

Componentes de una red neuronal

Paso 2: Pesos, suma ponderada y sesgo

- Cada conexión con cada neurona tiene un peso asociado
- Los pesos nos indican la importancia de cada input de cara al procesamiento de datos.
- Los pesos y el sesgo son los parámetros que las redes neuronales optimizan en el entrenamiento.

Componentes de una red neuronal

Paso 2: Pesos, suma ponderada y sesgo

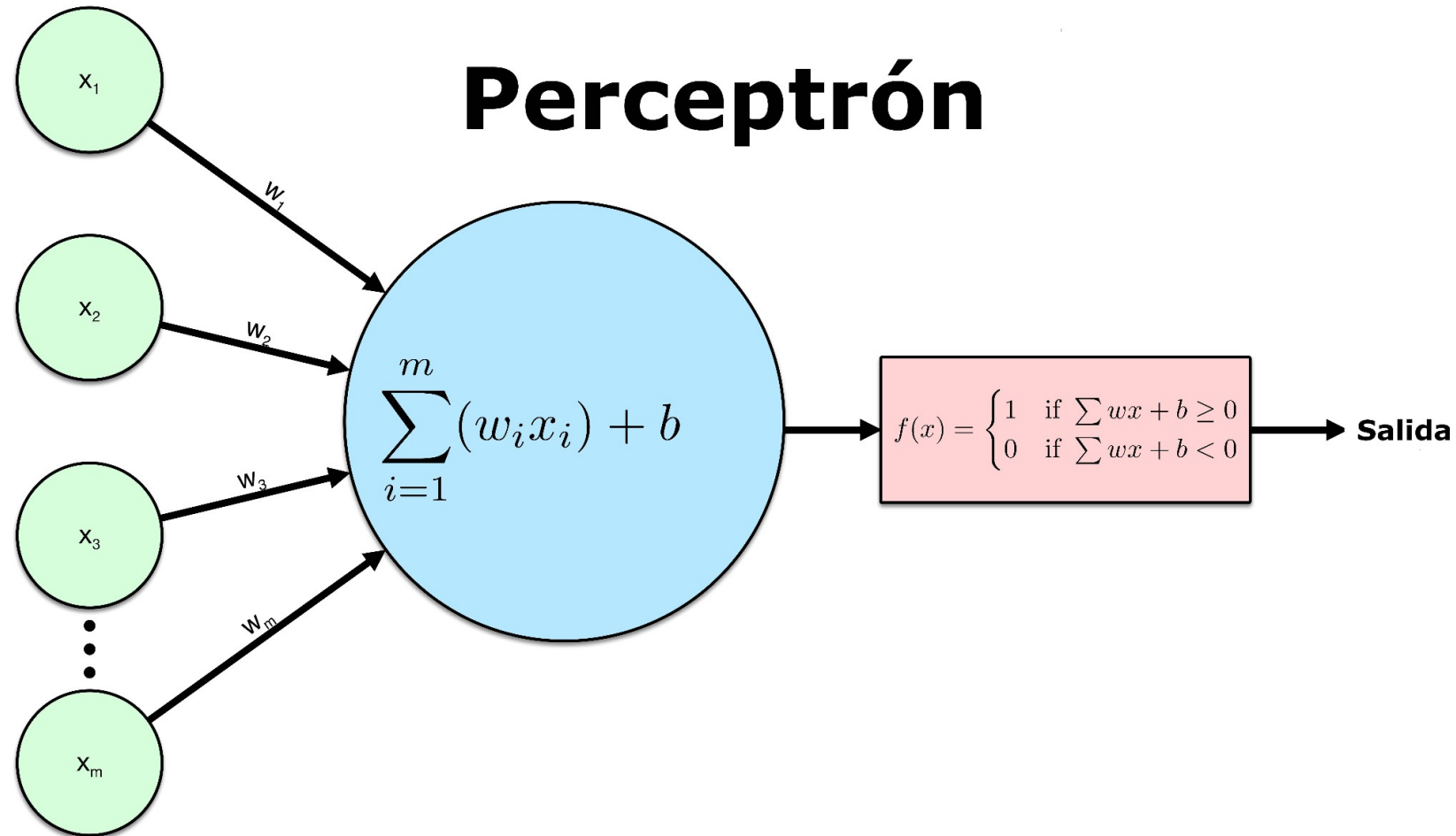
- La suma ponderada es el cálculo resultante de multiplicar cada dato input por su peso, sumar todos los valores, y finalmente sumarle un valor constante de sesgo.
- El sesgo es un componente vital para la capacidad de generalización de la red neuronal.

Componentes de una red neuronal

Función de activación:

- Una vez tenemos el valor resultante de la suma ponderada, se le aplica una función matemática conocida como función de activación. Esta función es vital para poder captar relaciones complejas en los datos.
- El resultado de la función de activación es el output que se envía a la siguiente capa de neuronas.

Componentes de una red neuronal

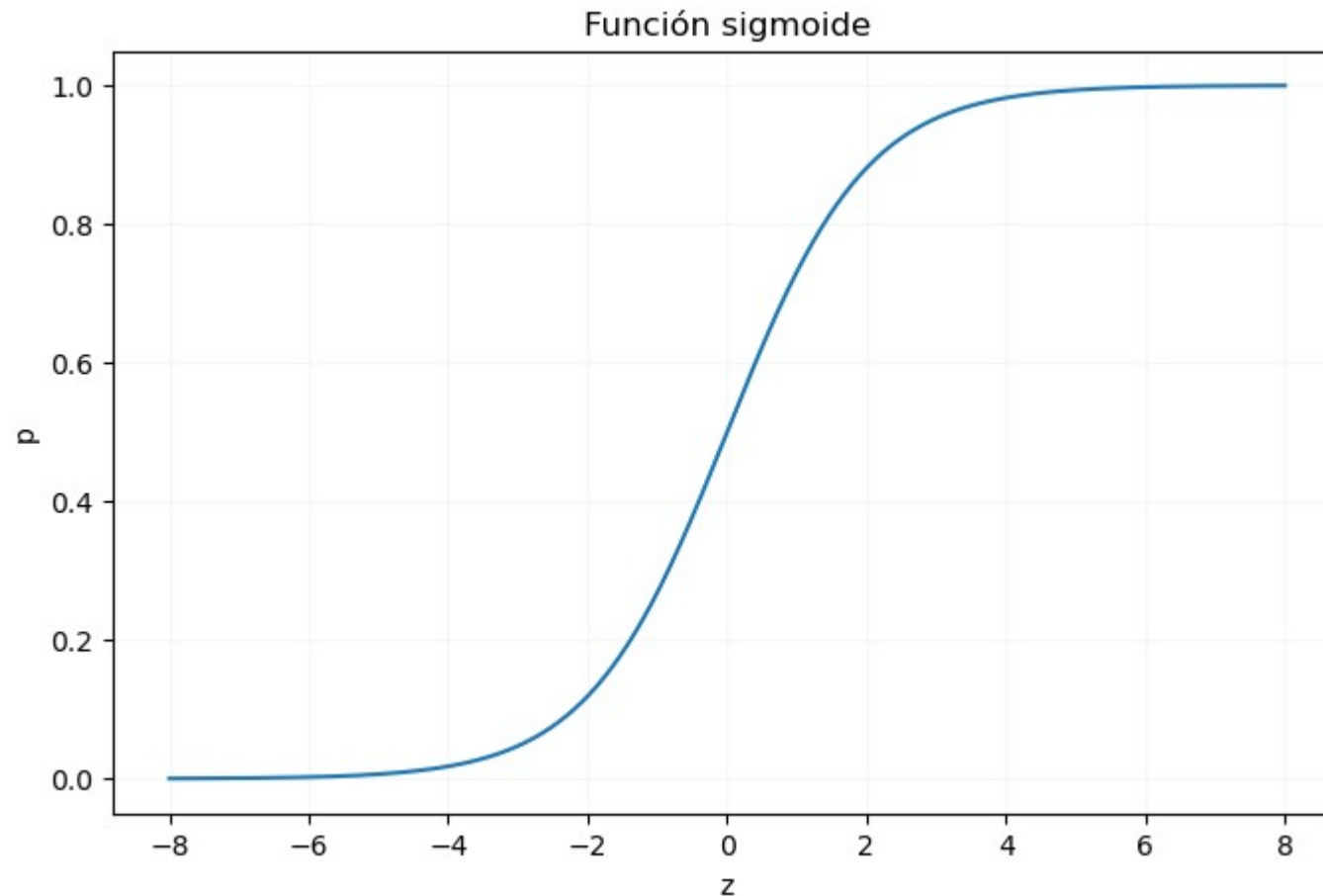


Componentes de una red neuronal

Funciones de activación:

- Sigmoides:
 - Misma función que en la regresión logística
 - Función suave (derivable) con forma de S, output entre 0 y 1.
 - Se utiliza en capas ocultas y siempre en la capa de salida de la clasificación binaria.

Componentes de una red neuronal



$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

Componentes de una red neuronal

Funciones de activación:

- Softmax:
 - Generalización de la función sigmoide para la clasificación multiclase, el output es un vector con valores entre 0 y 1, que suman 1 entre todos.
 - Se utiliza siempre en la capa de salida de la clasificación multiclase.

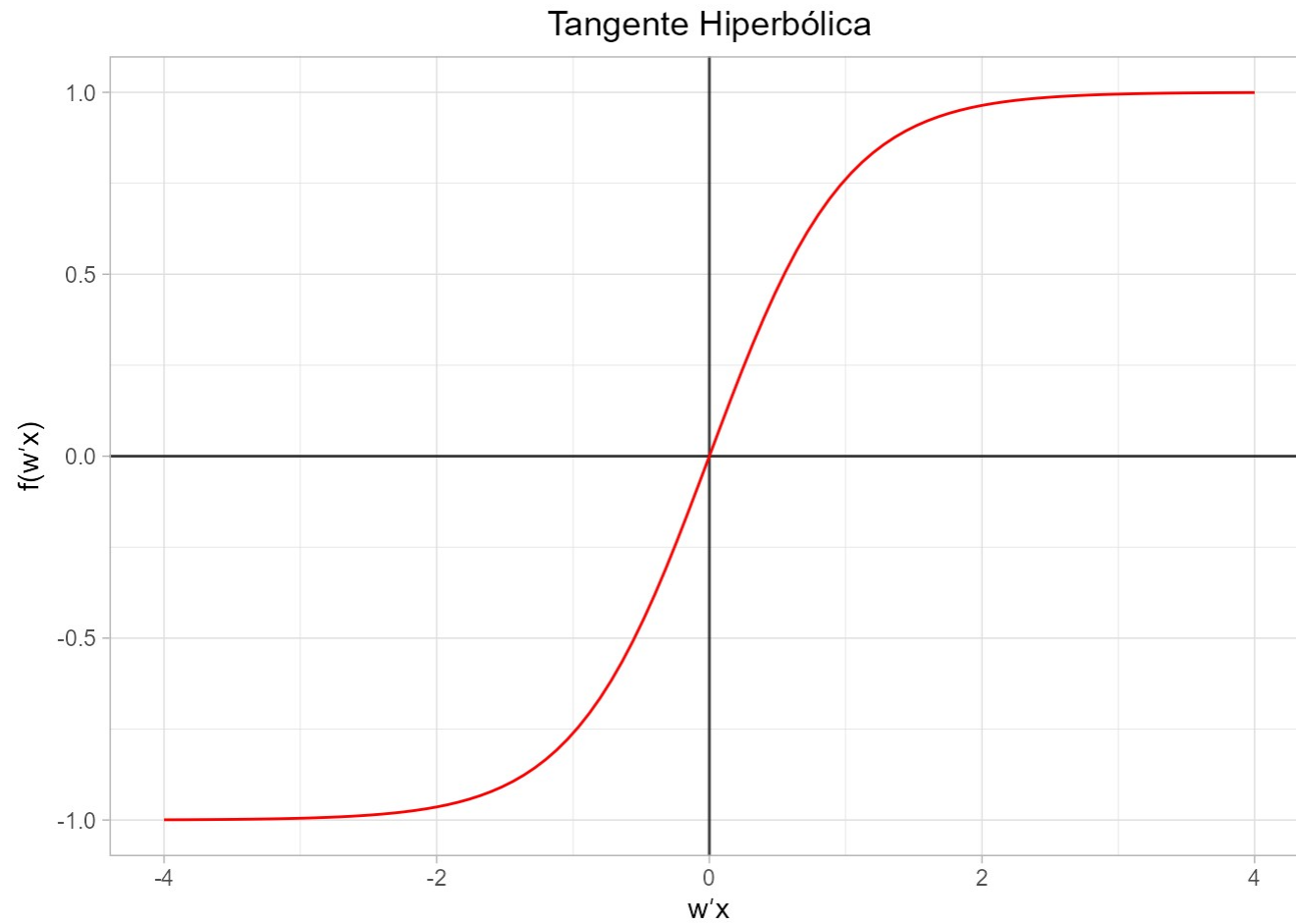
$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

Componentes de una red neuronal

Funciones de activación:

- Tangente hiperbólica ($\tanh$):
 - Función suave y con forma similar a la sigmoide, output entre -1 y 1.
 - Se utiliza en capas ocultas.

Componentes de una red neuronal



$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

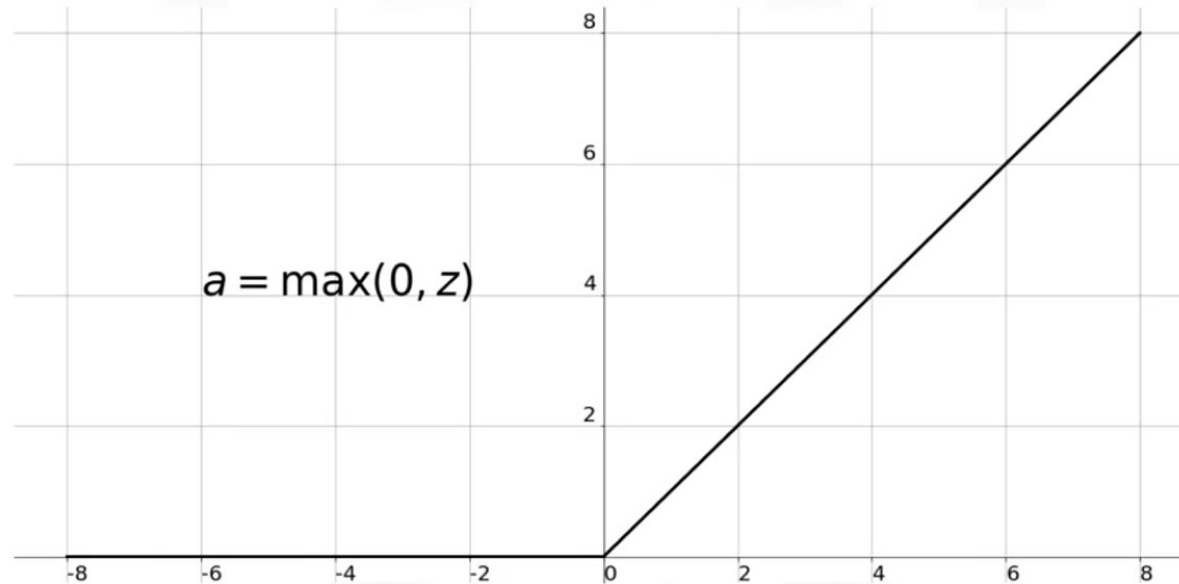
Componentes de una red neuronal

Funciones de activación:

- ReLU (Rectified Linear Unit):
 - Función no acotada, definida como 0 para valores negativos, y la identidad para valores positivos.
 - Se utiliza en capas ocultas.
 - Tiene muchas variantes, como por ejemplo la Leaky ReLU, que permite valores negativos.

Componentes de una red neuronal

ReLU Function



$$\text{ReLU}(z) = \max(0, z)$$

Proceso de entrenamiento de redes neuronales

- El número de capas y neuronas es fijo, así como su diseño (funciones de activación y conexiones).
- 4 pasos a repetir, análogo al proceso de entrenamiento de un algoritmo de Machine Learning:
 - Inicialización de los pesos (y sesgos) del modelo.
 - Forward propagation: Cálculo de resultados sobre el conjunto de entrenamiento.
 - Cálculo del error.
 - Backward propagation: Corrección de los pesos (y sesgos) en función del error obtenido.

Inicialización de los pesos (y sesgos) del modelo

- El primer paso es establecer los pesos y sesgos iniciales de la red antes de empezar el entrenamiento
- Técnicas principales:
 - Inicialización aleatoria con pesos pequeños.
 - Inicialización a partir de una red neuronal ya entrenada.
 - Técnicas de inicialización avanzadas: Xavier (varianza constante a través de las capas, frecuentemente utilizada con activación lineal o tanh), He (similar a Xavier pero adaptada a activación ReLU), LeCun...

Forward propagation

- Análogo al Machine Learning: Se toma el conjunto de datos de entrenamiento y para cada uno se realiza la secuencia de cálculos hasta obtener el resultado.
- En el caso de las redes neuronales, se va propagando el dato a través de la red neuronal con las conexiones, pesos y funciones de activación hasta obtener el resultado de la capa de salida.

Cálculo del error

- Análogo al Machine Learning: Se aplica la función de pérdida elegida (MAE, MSE, log-loss, cross-entropy, ...) para calcular el error de la salida del modelo para el dato de entrenamiento.
- El resultado de este cálculo nos mide cómo de lejos está la predicción del valor real, con lo que el objetivo de la red neuronal será ajustar los pesos y sesgos para minimizar este error.

Backward propagation

- El algoritmo utiliza el error calculado para actualizar los parámetros de manera que consigamos minimizar el error en la siguiente iteración.
- A la finalización del paso de backpropagation, se evalúa la función de pérdida sobre el validation set para comprobar la mejora del modelo en un conjunto de datos diferente al training set.

Backward propagation (Backpropagation)

- Un algoritmo de backpropagation utiliza el error calculado para actualizar los parámetros de manera que consigamos minimizar el error en la siguiente iteración.
- Hay diferentes técnicas, pero la técnica principal y más utilizada es Gradient Descent, con sus diferentes variantes.

Gradient descent

- Gradient descent se basa en calcular el gradiente de la función de pérdida respecto a cada parámetro.
- A nivel matemático, es la derivada parcial, que se interpreta como el cambio de la función de pérdida cuando se varía ese parámetro.
- Valores altos del gradiente indican que es necesario un mayor cambio del parámetro para mejorar el modelo, en el signo correspondiente.

Gradient descent

- Los gradientes se calculan para cada capa de la red neuronal, empezando por la capa de salida y utilizando la regla de la cadena para derivadas parciales.
- A medida que se van calculando los gradientes, se van corrigiendo los parámetros de cada capa de la red neuronal, por eso se conoce como propagación hacia atrás (backpropagation).

Gradient descent

Variaciones principales de gradient descent (GD):

- Stochastic GD (SGD)
- Mini-batch GD
- Momentum
- RMSprop
- Adam
- Adagrad
- ...

Posibles problemas en el proceso de entrenamiento

Desvanecimiento del gradiente:

- Ocurre cuando los gradientes de la función de error respecto a los pesos se vuelven muy muy pequeños, sobretodo para las capas iniciales de la red.
- Provoca que el ajuste de pesos sea muy pequeño, y por tanto un entrenamiento muy lento o subóptimo.
- Hay estrategias como cambiar funciones de activación a ReLu, cambiar la inicialización o usar métodos de backpropagation complejos como Adam o Adagrad.

Posibles problemas en el proceso de entrenamiento

Explosión del gradiente

- Ocurre cuando los gradientes de la función de error respecto a los pesos se vuelven muy muy grandes, sobretodo para las capas iniciales de la red.
- Provoca que el ajuste de pesos sea muy grande, y por tanto un entrenamiento inestable o subóptimo.
- Hay estrategias como cambiar funciones de activación a sigmoide o Tanh, cambiar la inicialización o usar métodos de backpropagation complejos como Adam o Adagrad.

Posibles problemas en el proceso de entrenamiento

Neuronas muertas

- Ocurre cuando ciertas neuronas tienen siempre peso 0, con lo cual dejan de activarse.
- Provoca la pérdida de neuronas, y por tanto pérdida de capacidad de aprendizaje de la red.
- Hay estrategias como usar dropout, cambiar funciones de activación a ReLu, cambiar la inicialización o usar métodos de backpropagation complejos como Adam o Adagrad.

Posibles problemas en el proceso de entrenamiento

Overfitting

- Ocurre cuando el modelo se ajusta demasiado a los datos de entrenamiento, perdiendo capacidad de generalización.
- La estrategia más habitual es dropout, consistente en desactivar aleatoriamente un conjunto de neuronas en cada iteración de entrenamiento.
- También se suele reducir el número de capas y neuronas como estrategia alternativa.

Posibles problemas en el proceso de entrenamiento

Underfitting

- Ocurre cuando el modelo se ajusta demasiado poco a los datos de entrenamiento, por lo tanto no obtiene resultados significativas.
- La estrategia más habitual es añadir neuronas, capas y conexiones a la red para darle más complejidad. También es importante intentar mejorar los datos de entrada.

Cierre

- Entrega de las actividades del Tema 4 - 05/05
- Siguiendo clase: 24/04 - Tema 4, parte 2
- Si tenéis consultas, me podéis escribir por privado
- ¿Dudas/preguntas?

